What is a Closure in JavaScript?

A **closure** is a feature in JavaScript where an **inner** function has access to the **outer** (enclosing) function's variables — **even after** the outer function has finished executing. This is possible because the inner function "closes over" the variables from its outer function, maintaining a reference to them.

In simpler terms, a closure allows a function to **remember the environment** (the lexical scope) in which it was **created**, even when it's executed outside of that environment.

Why is Closure Important?

Closures are important because they:

- **Encapsulate state**: They allow functions to "remember" and access variables from their outer scope, even after the outer function has finished executing. This can be useful for data hiding, maintaining private variables, or creating stateful functions.
- **Make functions more flexible**: Functions can behave dynamically by retaining context over variables in their lexical scope.
- **Enable advanced functional programming**: Closures are the backbone of many JavaScript patterns such as callbacks, event handlers, and functional programming techniques.

When Should You Use a Closure?

Closures are particularly useful in scenarios like:

- **Data privacy**: You want to "hide" certain data and expose only a specific interface.
- **Maintaining state**: You need to keep track of variables across function calls.
- **Callback functions**: You pass a function as an argument and want that function to remember variables from its original scope.
- **Factory functions**: You generate multiple functions, each with its own private state.

Example of Closures:

Private Variables (Data Encapsulation)

Closures can help hide variables and expose only certain parts of the functionality.

Example: Creating a function to handle bank account balances privately.

```
function createAccount(initialBalance) {
  let balance = initialBalance; // Private variable

  return {
    deposit(amount) {
      balance += amount;
      console.log(`Deposited: $${amount}. Current balance: $${balance}`);
    },
    withdraw(amount) {
      if (amount > balance) {
        console.log('Insufficient funds');
      } else {
        balance -= amount;
        console.log(`Withdrew: $${amount}. Current balance: $${balance}`);
      }
    },
    checkBalance() {
      console.log(`Current balance: $${balance}`);
    }
  };
}

const myAccount = createAccount(100);  // Initial balance is 100
myAccount.deposit(50);                 // Output: Deposited: $50. Current balance: $150
myAccount.withdraw(30);                // Output: Withdrew: $30. Current balance: $120
myAccount.checkBalance();              // Output: Current balance: $120
```

```
function createAccount(initialBalance) {
  let balance = initialBalance; // Private variable
  return {
    deposit(amount) {
      balance += amount;
      console.log(`Deposited: $${amount}. Current balance: $${balance}`);
    },
    withdraw(amount) {
      if (amount > balance) {
        console.log('Insufficient funds');
      } else {
        balance -= amount;
        console.log(`Withdrew: $${amount}. Current balance:
$${balance}`);
      }
    },
    checkBalance() {
      console.log(`Current balance: $${balance}`);
    }
  };
```

```
}

const myAccount = createAccount(100);  // Initial balance is 100
myAccount.deposit(50);                 // Output: Deposited: $50. Current
balance: $150
myAccount.withdraw(30);                // Output: Withdrew: $30. Current
balance: $120
myAccount.checkBalance();              // Output: Current balance: $120
```

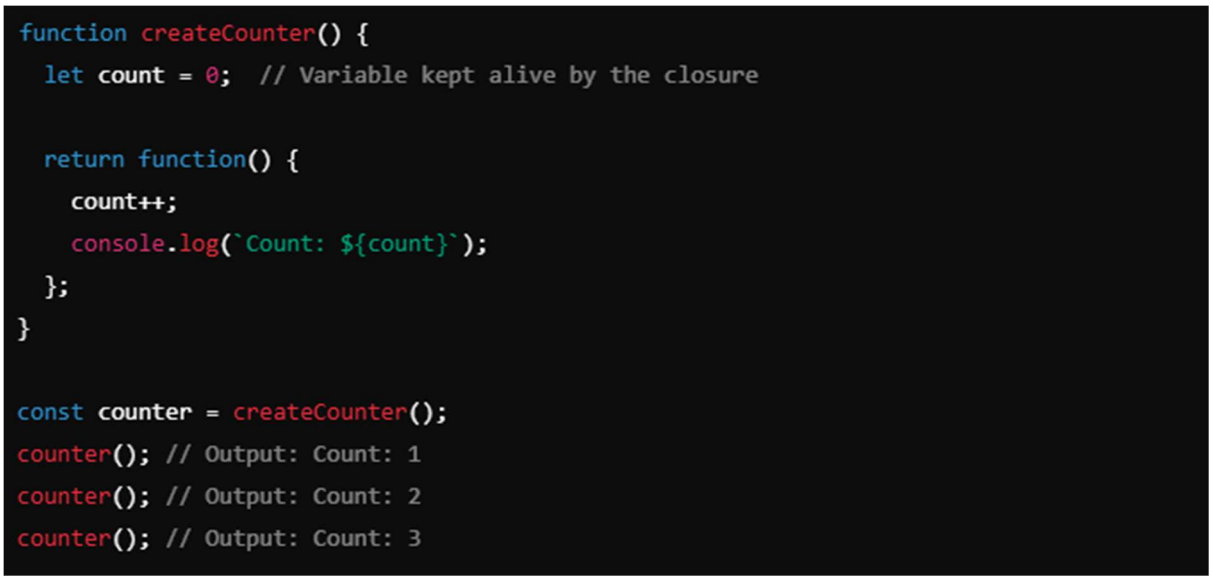
The balance variable is "private" because it's in the closure of the returned object (which contains deposit, withdraw, and checkBalance methods).

The outer function createAccount has finished executing, but the inner methods can still access balance through the closure.

Maintaining State Between Function Calls

Closures allow you to maintain state across multiple function invocations, which is useful for scenarios like counters or any function that requires stateful behavior.

Example: Simple counter.




```
function createCounter() {
  let count = 0;  // Variable kept alive by the closure

  return function() {
    count++;
    console.log(`Count: ${count}`);
  };
}

const counter = createCounter();
counter(); // Output: Count: 1
counter(); // Output: Count: 2
counter(); // Output: Count: 3
```

The count variable is remembered across function calls because the returned function forms a closure over count.

Every time counter() is called, it increments the count and logs the updated value.

Callback Functions with Closures

When using asynchronous operations or event listeners, closures are often essential for retaining references to certain variables.

Example: Using a closure inside a callback function

```
function fetchData(url) {
  const dataID = url.split('/').pop(); // Assume URL has an ID at the end

  // Simulate asynchronous fetch operation
  setTimeout(function() {
    console.log(`Fetched data for ID: ${dataID}`);
  }, 1000);  // Delayed execution
}

fetchData('https://example.com/data/123'); // Output: Fetched data for ID: 123
```


```
function fetchData(url) {
  const dataID = url.split('/').pop(); // Assume URL has an ID at the end

  // Simulate asynchronous fetch operation
  setTimeout(function() {
    console.log(`Fetched data for ID: ${dataID}`);
  }, 1000);  // Delayed execution
}

fetchData('https://example.com/data/123'); // Output: Fetched data for
ID: 123
```


Even though fetchData finishes executing almost instantly, the setTimeout function (a callback) still has access to the dataID variable due to closure. It prints the correct dataID after the timeout, despite the delay.

Factory Functions

Closures can be used to create functions dynamically, each with its own private variables.

Example: Discount Calculator Factory

```
function createDiscount(discountPercentage) {
  return function(price) {
    const discount   = price * (discountPercentage / 100);
    const finalPrice = price - discount;

    console.log(`Original Price: $${price}`);
    console.log(`Discount: $${discount.toFixed(2)}`);
    console.log(`Final Price after ${discountPercentage}% discount:
$${finalPrice.toFixed(2)}`);

    return finalPrice;
  };
}

const tenPercentDiscount    = createDiscount(10);
const twentyPercentDiscount = createDiscount(20);
const fiftyPercentDiscount  = createDiscount(50);

tenPercentDiscount(100);   // Apply 10% discount on $100
// Output:
// Original Price: $100
// Discount: $10.00
// Final Price after 10% discount: $90.00

twentyPercentDiscount(150); // Apply 20% discount on $150
// Output:
// Original Price: $150
// Discount: $30.00
// Final Price after 20% discount: $120.00

fiftyPercentDiscount(200);  // Apply 50% discount on $200
// Output:
// Original Price: $200
// Discount: $100.00
// Final Price after 50% discount: $100.00
```

The createDiscount factory function takes a discountPercentage as input and returns a function that calculates the discounted price for a given price.

Each generated function (like tenPercentDiscount, twentyPercentDiscount, and fiftyPercentDiscount) retains the specific discount percentage through a closure.

When you call the returned function with a price, it calculates the discount and logs the original price, the discount amount, and the final price after applying the discount.

This factory pattern is useful for creating discount calculators for different percentages, without having to repeat the discount logic for each case.

~ END ~